

Table of Contents

Introduction.....	2
HomeView and HomeController	3
Create Shared _Layout	6
Simplify Table.cshtml	11
Reset default launchURL.....	12
Run Project	13

Demo: Shop MVC 2, HomeView, shared _layout, navigation

Author: Baifan Zhou

Some explanations are created with the help of ChatGPT.

Introduction

This demo walks you through the creation and setup of a simple MVC web application using ASP.NET Core. The project demonstrates how to build views, controllers, and shared layouts, as well as how to configure the application for a smooth development experience. By the end of this demo, you'll have a functional MVC web application with a clean and organised structure.

HomeView and HomeController

In this section, you will create a HomeView and a HomeController to handle requests to the home page of your application. You'll learn how to:

- Create a subfolder Home under the Views folder and add an Index.cshtml file.
- Define the HomeController class to handle incoming HTTP requests and return the appropriate views.
- Understand how controllers and views work together in the MVC pattern, handling requests, executing actions, and rendering views.

Create Shared _Layout

Here, you'll set up a shared layout for your application:

- Create a subfolder Shared under the Views folder.
- Add a _Layout.cshtml file to define the common HTML structure for your pages, including headers, footers, and navigation.
- Use the @RenderBody() placeholder to inject content from individual views into the layout.
- Copy and modify layout and CSS files to maintain a consistent look and feel across your application.

Simplify Table.cshtml

This section guides you in simplifying the Table.cshtml view:

- Remove unnecessary headers and other elements to streamline the view.

Reset default launchURL

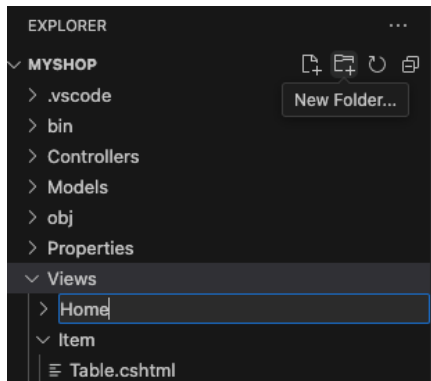
Learn how to reset the default launch URL for your application:

- Modify the .vscode/launch.json file to change the launch URI, ensuring the application starts on the desired page.

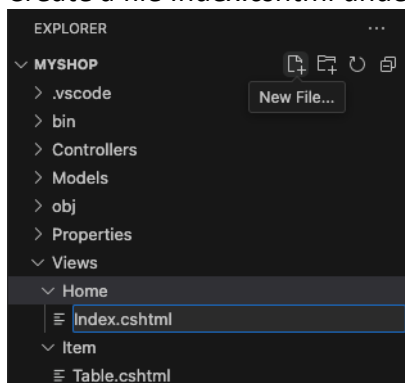
By following this demo, you will gain a solid understanding of setting up and configuring an ASP.NET Core MVC application, including creating views and controllers, setting up shared layouts, and simplifying view templates. Happy coding!

HomeView and HomeController

Create a subfolder “Home” under the folder Views



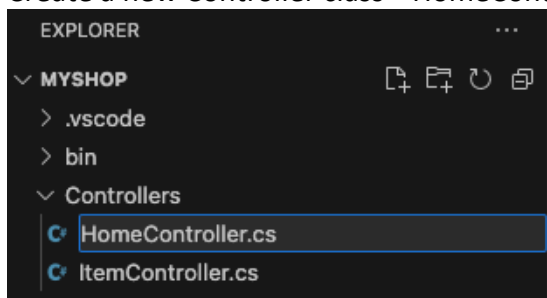
Create a file Index.cshtml under the Home folder



Change the Index.cshtml to this:

```
1  @{  
2      ViewData["Title"] = "Home Page";  
3  }  
4  
5  <div class="text-center">  
6      <h1 class="display-4">Welcome to My Shop</h1>  
7  </div>
```

Create a new Controller class “HomeController.cs” under the folder Controller



Change the file to:

```

1  using Microsoft.AspNetCore.Mvc;
2
3  namespace MyShop.Controllers
4  {
5      0 references
6      public class HomeController : Controller
7      {
8          // GET: /<controller>/
9          0 references
10         public IActionResult Index()
11         {
12             return View();
13         }
14     }
15 }

```

Detailed explanation:

MVC Architecture

The MVC pattern is a design pattern used to separate concerns within an application. It divides an application into three main components:

1. **Model:** Represents the data and the business logic.
2. **View:** Represents the presentation layer (UI).
3. **Controller:** Handles user input and interacts with the Model to render the appropriate View.

Controllers

A **Controller** is a class that handles incoming HTTP requests, processes them, and returns the appropriate response. Controllers are the heart of the MVC pattern, acting as the intermediary between the Model and the View.

- **Naming Convention:** Controllers typically end with the suffix "Controller". For example, HomeController, ItemController.
- **Location:** They are usually stored in the Controllers folder.

Actions

An **Action** is a method within a Controller that handles a specific request. Each Action method typically corresponds to a user interaction, such as viewing a list of items, displaying details of an item, or submitting a form.

- **Return Types:** Action methods usually return an IActionResult, which could be a View, JSON, a Redirect, or other result types.

HomeController.cs

The HomeController in this ASP.NET Core MVC application is responsible for handling requests to the home page of the website.

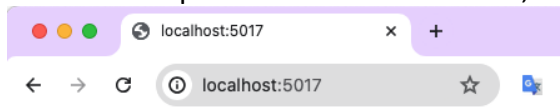
- **Controller Definition:** The HomeController class inherits from Controller, which is a base class in ASP.NET Core MVC that provides methods and properties for handling HTTP requests and returning responses.
- **Index Action Method:** The Index method is an action method. It's mapped to handle HTTP GET requests directed at the root URL (e.g., /Home/Index or / if routed as the default controller).

- **Returning a View:** When the Index action method is called, it returns a View result. This instructs the MVC framework to render a view named Index located in the Views/Home folder.

How the Controller and View Work Together

1. **Request Handling:** When a user navigates to the root URL or /Home/Index, the routing system directs the request to the Index action method of HomeController.
2. **Action Execution:** The Index action method executes and prepares any necessary data or simply decides which view to render. In this case, it just returns a View() without any model data.
3. **View Selection:** The MVC framework looks for a view file named Index.cshtml in the Views/Home directory.
4. **View Rendering:** The Index.cshtml view is rendered into HTML and sent back as the HTTP response to the user's browser.

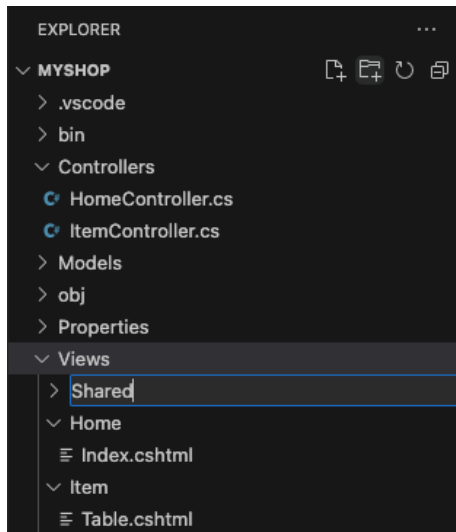
Now if you press F5, or click Menu Run → Start Debugging, and go to the home address, you will see a simple home view. In the next, we are going to create navigation.



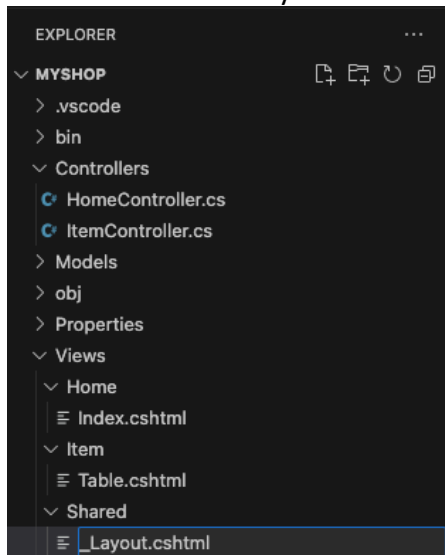
Welcome to My Shop

Create Shared _Layout

Create a subfolder “Shared” under the folder Views.



Create a file Razor Layout with the name “_Layout.cshtml” under the Shared folder



Copy the code in _Layout.cshtml from the FirstMVC project into the _Layout.cshtml file and Modify the code: change the stylesheet reference, change the nav-item, change the view title, delete the footer part

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>@ViewData["Title"] - FirstMVC</title>
7   <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
8   <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
9   <link rel="stylesheet" href="~/FirstMVC.styles.css" asp-append-version="true" />
10 </head>
11 <body>
12   <header>
13     <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
14       <div class="container-fluid">
15         <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">FirstMVC</a>
16         <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-
17           controls="navbarSupportedContent"
18           aria-expanded="false" aria-label="Toggle navigation">
19           <span class="navbar-toggler-icon"></span>
20         </button>
21         <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
22           <ul class="navbar-nav flex-grow-1">
23             <li class="nav-item">
24               <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
25             </li>
26             <li class="nav-item">
27               <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
28             </li>
29           </ul>
30         </div>
31       </div>
32     </header>
33     <div class="container">
34       <main role="main" class="pb-3">
35         @RenderBody()
36       </main>
37     </div>
38
39     <footer class="border-top footer text-muted">
40       <div class="container">
41         &copy; 2023 - FirstMVC - <a asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
42       </div>
43     </footer>
44     <script src="~/lib/jquery/dist/jquery.min.js"></script>
45     <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
46     <script src="~/js/site.js" asp-append-version="true"></script>
47     @await RenderSectionAsync("Scripts", required: false)
48   </body>
49 </html>

```

change

change

delete

Change to this:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>@ViewData["Title"] - My Shop</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
8 </head>
9 <body>
10   <header>
11     <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
12       <div class="container-fluid">
13         <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">My Shop</a>
14         <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-
15           controls="navbarSupportedContent"
16           aria-expanded="false" aria-label="Toggle navigation">
17           <span class="navbar-toggler-icon"></span>
18         </button>
19         <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
20           <ul class="navbar-nav flex-grow-1">
21             <li class="nav-item">
22               <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
23             </li>
24             <li class="nav-item">
25               <a class="nav-link text-dark" asp-area="" asp-controller="Item" asp-action="Table">Items</a>
26             </li>
27           </ul>
28         </div>
29       </div>
30     </header>
31     <div class="container">
32       <main role="main" class="pb-3">
33         @RenderBody()
34       </main>
35     </div>
36   </body>
37 </html>

```

Note: the routing and navigation follows the default pattern “Controller/Action” as specified as the default routing pattern in the Program.cs.

@RenderBody() is a placeholder where the content of the individual views will be injected.

Copy the file `_Layout.cshtml.css` from the FirstMVC project into the “Shared” folder

```
1  /* Please see documentation at https://docs.microsoft.com/aspnet/core/client-side/bundling-and-minification
2  for details on configuring this project to bundle and minify static web assets. */
3
4  a.navbar-brand {
5      white-space: normal;
6      text-align: center;
7      word-break: break-all;
8  }
9
10 a {
11     color: #0077cc;
12 }
13
14 .btn-primary {
15     color: #fff;
16     background-color: #1b6ec2;
17     border-color: #1861ac;
18 }
19
20 .nav-pills .nav-link.active, .nav-pills .show > .nav-link {
21     color: #fff;
22     background-color: #1b6ec2;
23     border-color: #1861ac;
24 }
25
26 .border-top {
27     border-top: 1px solid #e5e5e5;
28 }
29
30 .border-bottom {
31     border-bottom: 1px solid #e5e5e5;
32 }
33
34 .box-shadow {
35     box-shadow: 0 .25rem .75rem rgba(0, 0, 0, .05);
36 }
37
38 button.accept-policy {
39     font-size: 1rem;
40     line-height: inherit;
41 }
42
43 .footer {
44     position: absolute;
45     bottom: 0;
46     width: 100%;
47     white-space: nowrap;
48     line-height: 60px;
49 }
50
```

Create file “`_ViewImports.cshtml`” under the “Views” folder (not under “Shared”)

Add the following code into `_ViewImports.cshtml`

```
_Layout.cshtml  • _Layout.cshtml.css  • _ViewImports.cshtml
1  @using MyShop
2  @using MyShop.Models
3  @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
4
```

Note: It is special Razor view file allows you to define common statements applied to all other views in the application. It acts as a global import file for your views.
(Line3) is used to add tag helpers, which enables to create custom HTML elements (tags). Here it enables routing and navigation, e.g., the pattern of “Controller/Action”.

Create file Razor View Start under the Views folder

With the following code in the `_ViewStart.cshtml` (specifying the default layout):

It is a special Razor view file applied to all other views in the application. It acts as a global layout page for your views

```

1  @if
2      Layout = "_Layout";
3  }

```

Now we have the following files in the Views folder

```

Views
├── Home
│   └── Index.cshtml
├── Item
│   └── Table.cshtml
├── Shared
│   ├── _Layout.cshtml
│   ├── # _Layout.cshtml.css
│   ├── _ViewImports.cshtml
│   └── _ViewStart.cshtml

```

Detailed explanations:

Shared Folder in Views

The Shared folder within the Views directory is used to store views and other Razor files that are shared across multiple controllers and views in the application. This helps in avoiding duplication and maintaining a consistent look and feel throughout the application.

_Layout.cshtml

- **Purpose:** _Layout.cshtml is a special Razor file that defines a common layout (template) for your web pages. It typically includes the HTML structure, such as the <head>, <header>, <footer>, and any common scripts or styles.
- **Usage:** By using a layout, you ensure that all your pages share the same structure and design. Individual views will inject their content into the layout's @RenderBody() placeholder.
- **Location:** Views/Shared/_Layout.cshtml.

_ViewImports.cshtml

- **Purpose:** _ViewImports.cshtml is used to define common directives, namespaces, and tag helpers that should be available to all Razor views within the folder hierarchy where it is placed.
- **Usage:** This file helps to avoid repeating common directives across multiple views, promoting DRY (Don't Repeat Yourself) principles.
- **Location:** Typically located at the root of the Views folder but can also be placed in specific subfolders if different imports are needed for those views.
- **@using MyShop.Models:** Makes the MyShop.Models namespace available to all views, so you don't have to add it individually in each view file.
- **@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers:** Registers all the tag helpers provided by ASP.NET Core MVC, enabling the use of tag helpers like <partial>, <environment>, etc.

_ViewStart.cshtml

- **Purpose:** _ViewStart.cshtml is used to execute common code before rendering any Razor view. It typically sets the layout for the views.
- **Usage:** This file is executed before every view, allowing you to set properties like the layout dynamically.

- **Location:** Typically located at the root of the `Views` folder.
- `Layout = "_Layout";` specifies that all views should use the `_Layout.cshtml` file in the `Views/Shared` folder as their layout. This can be overridden in individual views if needed.

How They Work Together

1. **Layout:**
 - The `_Layout.cshtml` file in the `Views/Shared` folder defines the common layout structure for the application.
 - Individual views will be rendered within this layout using the `@RenderBody()` placeholder.
2. **Imports:**
 - The `_ViewImports.cshtml` file specifies common namespaces and tag helpers, ensuring they are available across all views without needing to be redefined.
3. **Start:**
 - The `_ViewStart.cshtml` file is executed before any view, typically setting the layout that the views should use. This ensures a consistent layout is applied unless explicitly overridden.

Example Workflow

1. **Request Handling:** A request is made to an action in a controller.
2. **Action Execution:** The controller action returns a view.
3. **View Rendering:**
 - Before rendering the view, `_ViewStart.cshtml` sets the layout to `_Layout.cshtml`.
 - `_ViewImports.cshtml` is processed, making common namespaces and tag helpers available.
 - The specific view is rendered within the `@RenderBody()` section of `_Layout.cshtml`.

Simplify Table.cshtml

Change the Table.cshtml file to this (remove the head and other information):

```
1  @model IEnumerable<MyShop.Models.Item>
2
3  <!DOCTYPE html>
4
5  <html>
6  <head>
7    <meta name="viewport" content="width=device-width" />
8    <link rel="stylesheet" href="//netdna.bootstrapcdn.com/bootstrap/3.0.3/css/bootstrap.min.css">
9    <title>My Shop</title>
10  </head>
11  <body>
12    <div class="container">
13      <h1>@ViewBag.CurrentViewName</h1>
14      <table class='table table-striped'>
15        <tr><th>Id</th><th>Name</th><th>Price</th><th>Description</th></tr>
16        @foreach (var item in Model)
17        {
18          <tr>
19            <td>@item.ItemId</td>
20            <td>@item.Name</td>
21            <td>@item.Price.ToString("0.00 NOK")</td>
22            <td></td>
23          </tr>
24        }
25      </table>
26    </div>
27  </body>
28  </html>
29
```

After change:

```
1  @model IEnumerable<MyShop.Models.Item>
2
3  <div class="container">
4    <h1>@ViewBag.CurrentViewName</h1>
5    <table class='table table-striped'>
6      <tr><th>Id</th><th>Name</th><th>Price</th><th>Description</th></tr>
7      @foreach (var item in Model)
8      {
9        <tr>
10          <td>@item.ItemId</td>
11          <td>@item.Name</td>
12          <td>@item.Price.ToString("0.00 NOK")</td>
13          <td></td>
14        </tr>
15      }
16    </table>
17  </div>
18
```

Reset default launchURL

Go to `.vscode/launch.json` and change the launch URI.

```
() launch.json X
.vscode > {} launch.json > ...
1  {
2    // Use IntelliSense to learn about possible attributes.
3    // Hover to view descriptions of existing attributes.
4    // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5    "version": "0.2.0",
6    "configurations": [
7      {
8        "name": ".NET Core Launch (web)",
9        "type": "coreclr",
10       "request": "launch",
11       "preLaunchTask": "build",
12       "program": "${workspaceFolder}/bin/Debug/net10.0/MyShop.dll",
13       "args": [],
14       "cwd": "${workspaceFolder}",
15       "stopAtEntry": false,
16       "serverReadyAction": {
17         "action": "openExternally",
18         "pattern": "\\bNow listening on:\\s+(https?://\\S+)",
19         "uriFormat": "%s/item/table",
20       },
21       "env": {
22         "ASPNETCORE_ENVIRONMENT": "Development"
23       },
24       "sourceFileMap": {
25         "/Views": "${workspaceFolder}/Views"
26       }
27     }
28   ]
29 }
```

After change:

```
() launch.json X
.vscode > {} launch.json > ...
1  {
2    // Use IntelliSense to learn about possible attributes.
3    // Hover to view descriptions of existing attributes.
4    // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5    "version": "0.2.0",
6    "configurations": [
7      {
8        "name": ".NET Core Launch (web)",
9        "type": "coreclr",
10       "request": "launch",
11       "preLaunchTask": "build",
12       "program": "${workspaceFolder}/bin/Debug/net8.0/MyShop.dll"
13       "args": [],
14       "cwd": "${workspaceFolder}",
15       "stopAtEntry": false,
16       "serverReadyAction": {
17         "action": "openExternally",
18         "pattern": "\\bNow listening on:\\s+(http?://\\S+)",
19         "uriFormat": "%s"
20       },
21       "env": {
22         "ASPNETCORE_ENVIRONMENT": "Development"
23       },
24       "sourceFileMap": {
25         "/Views": "${workspaceFolder}/Views"
26       }
27     }
28   ]
29 }
```

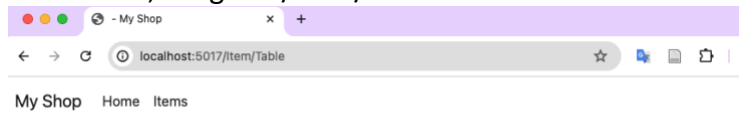
Run Project

Now start debugging (press F5) , the default page becomes /Home/Index



Welcome to My Shop

Click Items, we go to /Item/Table



List of Shop Items

Id	Name	Price	Description
1	Pizza	60.00 NOK	
2	Fried Chicken Leg	15.00 NOK	

Congratulations! You have added HomeView and Navigation to the project 😊